

Subject: CS336 Assignment 1, Optimizer

Date: from 2026/8/8 to 2026/8/11

Contents

Optimizer	1
------------------	----------

Optimizer

优化器的本质

在训练大模型时，我们的目标是寻找一组参数 θ ，使得损失函数 $L(\theta)$ 最小。优化器就是更新参数的策略。

- 输入. 当前的梯度 $g_t = \nabla L(\theta_t)$ ，告诉我们要往哪走；
- 状态. 历史的信息；
- 输出. 下一步的参数 θ_{t+1} 。

优化器的历史演变

1. SGD (随机梯度下降)：只看当下的“鲁莽汉”最基础的更新策略，没有任何记忆，完全依赖当前梯度。

$$\theta_{t+1} = \theta_t - \eta \cdot g_t$$

其中 η 是学习率（步长）， g_t 是当前时刻的梯度。痛点：

- 遇到峡谷（震荡）。如果梯度方向变化剧烈（如在一个狭长的山谷里），SGD 会在两壁之间反复横跳，收敛极慢；
- 遇到平原（停滞）。如果梯度很小（鞍点），更新量 $\eta \cdot g_t$ 会趋近于 0，模型走不动。

2. Momentum (动量法)：引入惯性的“铁球”为抑制震荡，模拟物理世界中的动量，让参数更新不仅依赖当前梯度，还保留一部分之前的速度。

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad \theta_{t+1} = \theta_t - \eta \cdot m_t$$

- m_t ：一阶动量（梯度的指数移动平均）；
- β_1 ：摩擦系数（通常 0.9），意味着保留 90% 的历史速度，只听 10% 的当前指挥。进化点：
 - 冲过平原（局部最优解）。即使当前梯度 g_t 为 0，靠着历史惯性 m_{t-1} ，球还能继续滚；
 - 抑制震荡。震荡方向的梯度正负相消，而主路方向的梯度不断累积，速度越来越快。

3. RMSProp (均方根传播)：自带路况适应的“越野车”Momentum 解决了方向问题，但没解决步长问题。我们希望：陡峭处步子小点（防炸），平坦处步子大点（提速）。

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{v_t} + \epsilon} \cdot g_t$$

- v_t ：二阶动量（梯度平方的移动平均，反映梯度的“能量”或“波动程度”）；
- $\frac{1}{\sqrt{v_t}}$ ：自适应缩放系数——梯度越大分母越大、步长越小（自动刹车），梯度越小步长越大（自动加油）；
- 进化点. 实现了参数级的自适应学习率，不同参数可以有不同的更新速度。

4. Adam (Adaptive Moment Estimation)：集大成者 Adam 是 Momentum 和 RMSProp 的结合体：既有惯性（一阶矩），又有自适应步长（二阶矩），并加入了偏差修正。

1. 算一阶矩（方向）： $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$ ；
2. 算二阶矩（力度）： $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ ；

3. 偏差修正（解决冷启动）: $\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \hat{v}_t = \frac{v_t}{1 - \beta_2^t};$
4. 更新参数: $\theta_{t+1} = \theta_t - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$

AdamW

Adam 看起来已经完美了，但在很长一段时间里，它在 CV 和 NLP 的最终泛化能力上都不如 SGD + Momentum. 直到 2017 年，人们发现 Adam 在处理权重衰减（Weight Decay）时存在严重的逻辑错误。

1. L2 正则化 vs. 权重衰减 在 SGD 中，这两者是等价的：

- L2 正则. 在 Loss 后加一项 $\frac{1}{2}\lambda\|\theta\|^2$;
- 求导后. 梯度变成 $g_t + \lambda\theta$;
- SGD 更新.

$$\theta_{t+1} = \theta_t - \eta(g_t + \lambda\theta_t) = \theta_t - \eta g_t - \underbrace{\eta\lambda\theta_t}_{\text{weight decay}}$$

可以看到，L2 正则化最终导出了权重衰减项。

2. Adam 的“耦合”灾难 如果把 L2 正则化（ $g_t + \lambda\theta$ ）直接塞进 Adam 的更新公式：

$$\theta_{t+1} = \theta_t - \eta \frac{M_t(g_t + \lambda\theta)}{\sqrt{V_t(g_t + \lambda\theta)} + \epsilon}$$

注意分母上的 $\sqrt{V_t}$ ：

- 实际的衰减力度变成了 $\frac{\eta\lambda}{\sqrt{v_t}}$;
- 后果.
 - 当 v_t 很大时（梯度变化剧烈，如陡峭区域）：分母大，衰减力度变小. 这很不合理——梯度大的参数往往数值也大，恰恰需要更强的正则化来抑制过拟合，Adam 却反而保护了它；
 - 当 v_t 很小时（平坦区域）：分母小，衰减力度变大，可能会误杀重要的细微特征.

权重衰减

在大模型（LLM）预训练中，权重衰减是必须开启的，原因有三点：

1. 防止“记忆过载”（Overfitting）. 如果权重非常大，模型就会变得非常“敏感”：输入里一丁点噪声，经过大权重放大都会导致输出剧变，这说明模型在“死记硬背”训练数据. 权重衰减强迫模型用更小的权重去解决问题，从而逼模型学习普适规律，而不是记住噪声；
2. 提高数值稳定性（Numerical Stability）. 大模型有上百层，如果每层权重都很大，信号在传递过程中会指数级膨胀，最终导致 NaN（数值溢出）. 权重衰减像一个“限压阀”，把参数控制在温和范围内，保住模型的命；
3. 增加泛化能力（Occam's Razor）. 根据奥卡姆剃刀原理，如果两个模型都能解释数

据，我们倾向于选择更简单的那个。权重更小的模型在数学上等价于更简单的函数，面对从未见过的测试数据时表现更稳。

算法全流程

1. 为什么要有偏置校正？Adam 的 m_t （一阶动量）、 v_t （二阶动量）是指数移动平均：

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \quad m_0 = 0, v_0 = 0$$

迭代早期 t 很小时， m_t, v_t 会偏向 0，估计有偏差。例如 $t = 1$ ：

$$m_1 = (1 - \beta_1) g_1, \quad v_1 = (1 - \beta_2) g_1^2$$

真实梯度均值 $\approx g_1$ ，但 m_1 被乘了 $(1 - \beta_1)$ ，数值偏小。偏置校正就是把这个初始被压低的动量估计还原回去：

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

$\hat{m}_t, \hat{v}_t$ 是校正之后无偏的动量估计。

2. 把标准 Adam 更新式变形，得到 **step_size** 标准参数更新为

$$\theta_t = \theta_{t-1} - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

代入校正定义 $\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$ 、 $\sqrt{\hat{v}_t} = \frac{\sqrt{v_t}}{\sqrt{1 - \beta_2^t}}$ ：

$$\frac{\hat{m}_t}{\sqrt{\hat{v}_t}} = \frac{m_t / (1 - \beta_1^t)}{\sqrt{v_t} / \sqrt{1 - \beta_2^t}} = \frac{m_t}{\sqrt{v_t}} \cdot \frac{\sqrt{1 - \beta_2^t}}{1 - \beta_1^t}$$

于是原式改写为

$$\theta_t = \theta_{t-1} - \underbrace{\eta \cdot \frac{\sqrt{1 - \beta_2^t}}{1 - \beta_1^t}}_{\text{step_size}} \cdot \frac{m_t}{\sqrt{v_t} + \epsilon}$$

在 `step()` 函数里，每一个变量都对应算法的一步；下面是与之对应的 PyTorch 原地操作（共 5 步，最后一步是权重衰减）：

```
exp_avg.mul_(beta1).add_(grad, alpha=1-beta1)
exp_avg_sq.mul_(beta2).addcmul_(grad, grad, value=1-beta2)
step_size = lr * (bias_correction2_sq / bias_correction1)
p.addcdiv_(exp_avg, denom, value=-step_size)
p.add_(p, alpha=-lr * wd)
```